

October 18

Problem 1: Roman to Integer

Problem Statement:

Given a Roman numeral, convert it to an integer. Roman numerals are represented by seven different symbols: I, V, X, L, C, D, M.

Link to problem:

<https://leetcode.com/problems/roman-to-integer/description/>

Example 1:

- Input: `s = "III"`
- Output: `3`

Example 2:

- Input: `s = "MCMXCIV"`
 - Output: `1994`
-

Solution:

```
import java.util.*;
```

```
class Solution {
    public int romanToInt(String s) {
        Map<Character, Integer> roman = new HashMap<>();
        roman.put('I', 1);
        roman.put('V', 5);
        roman.put('X', 10);
        roman.put('L', 50);
        roman.put('C', 100);
        roman.put('D', 500);
        roman.put('M', 1000);

        int result = 0;
        for (int i = 0; i < s.length(); i++) {
            int current = roman.get(s.charAt(i));
            if (i < s.length() - 1 && current < roman.get(s.charAt(i + 1))) {
                result -= current;
            } else {
                result += current;
            }
        }
        return result;
    }
}
```

```
}
```

Explanation:

- **Roman Numeral Mapping:** Map each Roman numeral to its corresponding integer value.
 - **Iterating through the String:** For each character, check if it's smaller than the next one (special cases like IV or IX), if yes, subtract its value; otherwise, add its value.
 - **Final Output:** Return the computed result.
-

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Problem 2: Find the Index of the First Occurrence in a String

Problem Statement:

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or -1 if `needle` is not part of `haystack`.

Link to problem:

<https://leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string/description/>

Example 1:

- Input: `haystack = "hello", needle = "ll"`
- Output: 2

Example 2:

- Input: `haystack = "aaaaa", needle = "bba"`
 - Output: -1
-

Solution:

```
class Solution {
    public int strStr(String haystack, String needle) {
        if (needle.isEmpty()) return 0;
        int hLen = haystack.length();
        int nLen = needle.length();

        // If needle is longer than haystack, it's not possible to find it.
```

```

    if (nLen > hLen) return -1;

    // Slide over haystack with a window of size equal to needle length.
    for (int i = 0; i <= hLen - nLen; i++) {
        int j = 0;
        // Compare each character of haystack with needle
        while (j < nLen && haystack.charAt(i + j) == needle.charAt(j)) {
            j++;
        }
        // If we've gone through the whole needle, a match was found
        if (j == nLen) {
            return i;
        }
    }

    return -1; // If no match is found, return -1.
}

```

Explanation:

- **Edge case:** If the `needle` is empty, return 0 since an empty string is always found at the start of any string.
 - **Sliding window:** Traverse the `haystack` with a sliding window of the same size as the `needle`. At each position in the `haystack`, check character by character if the `needle` matches.
 - **Inner loop:** Compare the current window of characters from the `haystack` with the `needle`. If all characters match (`j == nLen`), return the starting index `i`.
 - **Final check:** If no match is found after traversing the `haystack`, return -1.
-

Example :

Example 1:

- **Input:**
 - `haystack = "hello"`
 - `needle = "ll"`
- **Steps:**
 - In the first iteration (`i = 0`), the substring "he" does not match "ll".
 - In the second iteration (`i = 1`), the substring "el" does not match "ll".
 - In the third iteration (`i = 2`), the substring "ll" matches "ll", so the function returns 2.

Example 2:

- **Input:**
 - `haystack = "aaaaa"`

- `needle = "bba"`
- **Steps:**
 - None of the substrings match "bba". The function checks all windows in the `haystack` but finds no match, so it returns -1.

Example 3:

- **Input:**
 - `haystack = "hello"`
 - `needle = ""`
 - **Steps:**
 - The `needle` is an empty string, so the function returns 0 immediately.
-

Time Complexity:

- $O(n * m)$ where n is the length of the haystack and m is the length of the needle. We perform a linear scan of the haystack and compare up to m characters for each possible starting index.

Space Complexity:

- $O(1)$ since no extra space proportional to the input size is used, only a few integer variables are needed.
-

Problem 3: Valid Anagram

Problem Statement:

Given two strings s and t , return true if t is an anagram of s , and false otherwise.

Link to problem:

<https://leetcode.com/problems/valid-anagram/description/>

Example 1:

- Input: $s = \text{"anagram"}, t = \text{"nagaram"}$
- Output: `true`

Example 2:

- Input: $s = \text{"rat"}, t = \text{"car"}$
 - Output: `false`
-

Solution:

```
import java.util.*;

class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;

        int[] count = new int[26];
        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
        }

        for (int c : count) {
            if (c != 0) return false;
        }

        return true;
    }
}
```

Explanation:

- **Frequency Count:** Use an array to count the frequency of characters in both s and t.
- **Comparison:** After processing both strings, if the array contains only zeros, the two strings are anagrams.

Time Complexity: $O(n)$ **Space Complexity: $O(1)$**
